

Mini-projet 02 — Le constructeur de commandes

Module A Notions : **paramètres positionnels / nommés** · **valeurs par défaut** · ***args** · ****kwargs** · Durée : 1 h 30 – 2 h · Parcours Python — Ednir

Pourquoi ce mini-projet ?

Dans ta boîte à outils, chaque plugin devra **fabriquer une commande** à partir d'options variables : parfois 2 ports, parfois 50 ; parfois un timeout, parfois pas ; parfois des options exotiques qu'on n'avait pas prévues. Une fonction rigide (nombre d'arguments fixe) ne tient pas. Ce mini-projet t'apprend à écrire des fonctions **flexibles** — c'est LA base d'une API propre.

1. Positionnel vs nommé

Quand tu appelles une fonction, tu peux passer les arguments de deux façons :

```
def scan(cible, port):
    print(cible, port)

scan("10.0.0.5", 22)          # positionnel : l'ORDRE décide qui est quoi
scan(port=22, cible="10.0.0.5") # nommé : le NOM décide, l'ordre n'importe plus
```

À retenir : positionnel = « le 1er argument va au 1er paramètre ». Nommé = « je dis explicitement **port=22** ». Les deux donnent le même résultat ici.

2. Les valeurs par défaut

Un paramètre peut avoir une **valeur par défaut** : si l'appelant ne la fournit pas, Python utilise la valeur prévue.

```
def scan(cible, timeout=2, verbose=False):
    print(cible, timeout, verbose)

scan("10.0.0.5")          # -> 10.0.0.5 2 False (défauts utilisés)
scan("10.0.0.5", timeout=5) # -> 10.0.0.5 5 False (on écrase 1 défaut)
scan("10.0.0.5", verbose=True) # -> 10.0.0.5 2 True
```

Règle d'ordre : les paramètres **sans** défaut viennent **toujours avant** ceux **avec** défaut.

```
def scan(cible, timeout=2) ✓ def scan(timeout=2, cible) ✗ SyntaxError
```

3. ⚠ Le piège n°1 : le défaut mutable

C'est le piège classique de Python, et il est vicieux. **Ne mets jamais une liste (ou un dict) vide comme valeur par défaut.**

```
# ✗ NE FAIS JAMAIS ÇA
def ajouter(element, liste=[]):
    liste.append(element)
    return liste

ajouter(1) # -> [1] ok...
ajouter(2) # -> [1, 2] QUOI ?! la liste a gardé le 1 !
```

Pourquoi ? La liste par défaut **n'est créée qu'une seule fois**, au moment où Python lit le **def**. Tous les appels partagent donc **la même** liste. C'est exactement le problème « une collection ne doit pas être passée en paramètre, elle doit être créée à l'intérieur ».

```
# ✓ LA BONNE FAÇON : défaut = None, on crée la liste DANS la fonction
def ajouter(element, liste=None):
    if liste is None:
        liste = [] # une liste NEUVE à chaque appel
    liste.append(element)
    return liste
```

4. `*args` — un nombre variable d'arguments positionnels

Le `*` devant un paramètre dit : « ramasse **tous** les arguments positionnels restants et mets-les dans un **tuple** ».

```
def scan(cible, *ports):
    print(cible)    # 10.0.0.5
    print(ports)    # (22, 80, 443)    <- un tuple

scan("10.0.0.5", 22, 80, 443)
```

Le nom `args` est une convention, pas une obligation : c'est le `*` qui fait la magie. Ici on l'a appelé `ports` parce que c'est plus parlant. Tu peux boucler dessus : `for p in ports:` .

5. `**kwargs` — un nombre variable d'arguments nommés

Le `**` ramasse tous les arguments **nommés** restants dans un **dictionnaire**.

```
def scan(cible, **options):
    print(cible)    # 10.0.0.5
    print(options)  # {'retry': 3, 'proxy': 'off'}    <- un dict

scan("10.0.0.5", retry=3, proxy="off")
```

Moyen mnémo : `*args` = un sac de valeurs (tuple), `**kwargs` = un sac de paires clé/valeur (dict). `kwargs` = « *keyword arguments* ».

6. L'ordre canonique des paramètres

Quand tu combines tout, Python impose un ordre strict dans la signature :

```
def f( positionnels , *args , nommés_avec_défaut , **kwargs )
```

```
def construire(cible, *ports, timeout=2, **options):
    ...
# cible      -> positionnel obligatoire
# *ports     -> tous les ports en plus (tuple)
# timeout    -> nommé avec défaut (vient APRÈS *ports = "keyword-only")
# **options-> le reste des options nommées (dict)
```

Détail pro : un paramètre placé **après** `*args` (comme `timeout` ci-dessus) devient **keyword-only** : on ne peut le passer que nommé (`timeout=5`), plus jamais par position. C'est voulu, ça rend l'appel lisible.

🎯 Ta mission

Objectif : écrire `construire_commande(...)` qui assemble une **chaîne de commande** à partir d'arguments flexibles. (On construit juste une **chaîne de texte** — on n'exécute rien. Uniquement pour ton lab.)

Signature imposée :

```
def construire_commande(cible, *ports, timeout=2, verbose=False, **options):
```

Comportement attendu : la fonction renvoie (avec `return` , pas `print`) une chaîne construite ainsi :

- commence par `scan <cible>` ;
- s'il y a des `ports` , ajoute `--ports 22,80,443` (les ports séparés par des virgules) ;
- ajoute toujours `--timeout <timeout>` ;
- si `verbose` est vrai, ajoute `--verbose` ;
- pour chaque option dans `**options` , ajoute `--clé valeur` .

Tests que ton code doit passer :

```
construire_commande("10.0.0.5")
# -> "scan 10.0.0.5 --timeout 2"

construire_commande("10.0.0.5", 22, 80, 443, timeout=5, verbose=True)
# -> "scan 10.0.0.5 --ports 22,80,443 --timeout 5 --verbose"

construire_commande("10.0.0.5", 22, retry=3, proxy="off")
# -> "scan 10.0.0.5 --ports 22 --timeout 2 --retry 3 --proxy off"
```

✓ Critères de réussite

- La fonction utilise `*ports` ET `**options` .
- `timeout` et `verbose` ont des valeurs par défaut.
- Les 3 tests ci-dessus renvoient exactement les chaînes attendues.
- La fonction fait `return` (elle renvoie la chaîne), elle ne se contente pas d'un `print` .

Si tu bloques — indices progressifs

Étape	Indice
Assembler morceau par morceau	Pars d'une liste de bouts : <code>parties = ["scan", cible]</code> , puis <code>parties.append(...)</code> au fur et à mesure, et à la fin <code>"</code> <code>.join(parties)</code> pour tout coller avec des espaces.
Les ports en <code>22,80,443</code>	Tu as un tuple de nombres. <code>"", ".join(...)</code> ne marche que sur des chaînes → il faut d'abord convertir chaque port en str (une compréhension du mini-projet 01 : <code>[str(p) for p in ports]</code>).
N'ajouter les ports que s'il y en a	<code>if ports:</code> — un tuple vide est « faux ».
Le drapeau <code>--verbose</code>	<code>if verbose:</code> → alors <code>parties.append("--verbose")</code> .
Les options <code>**options</code>	C'est un dict → <code>for cle, valeur in options.items():</code> puis ajoute <code>f"--{cle} {valeur}"</code> .

Où ranger ce mini-projet : crée `mini-projets/02-constructeur-commandes/commande.py` .

Quand c'est prêt : `git add .` → `git commit -m "Mini-projet 02 : constructeur de commandes"` → `git push` , puis dis-moi « c'est poussé ». Je relis et on affine.